

PATTERN MATCHING

Giovedì 2 aprile 2026



PATTERN MATCHING PROBLEM

- Supponiamo che siano assegnate due stringhe: il testo T e il pattern P , rispettivamente di lunghezza $n > 0$ e $m > 0$, tipicamente con $n \gg m$.
- Il problema del pattern matching consiste nel trovare, se esiste, un'occorrenza del pattern P nel testo T , ovvero se esiste un valore i tale che $P = T[i \dots i + m - 1]$. Se tale valore i esiste allora diremo che P ha un match alla posizione i del testo T .



- Esiste una variante del problema che ha come obiettivo quello di contare o individuare tutte le occorrenze del pattern nel testo.



ALCUNE APPLICAZIONI...

Find & Replace

Simple Advanced

Find: search

Replace:

Replace All Replace Replace & Find Previous Next

indexof java algorithm - Ce

pdf thesis_Holley.pdf Posta i

GOOGLE

Google o digita un URL

Apache SpamAssassin

Posta in arrivo... Your Projects Hash Tables DeepL Tradutt...

Personalizza

(Matematica.blu 2.....pdf) Ugo Amaldi - L'A....pdf Untitled (Draft) (1).pdf Mostra tutto



STRATEGIA BRUTE-FORCE (O INGENUA)

- Controlla il match del pattern a partire da ogni posizione del testo.

i	j	i+j	0	1	2	3	4	5	6	7	8	9	10
txt →			A	B	A	C	A	D	A	B	R	A	C
0	2	2	A	B	R	A	← pat						
1	0	1		A	B	R	A	entries in red are mismatches					
2	1	3			A	B	R	A	entries in gray are for reference only				
3	0	3				A	B	R	A	entries in black match the text			
4	1	5					A	B	R	A			
5	0	5						A	B	R	A		
6	4	10							A	B	R	A	
return i when j is M													
										↑ match			



STRATEGIA BRUTE-FORCE

i	j	i+j	0	1	2	3	4	5	6	7	8	9	10
			A	B	A	C	A	D	A	B	R	A	C
4	3	7					A	D	A	C	R		
5	0	5						A	D	A	C	R	

```

void pattern_matching_naive(string pattern, string text)
{
    int M = pattern.size();
    int N = text.size();
    for (int i = 0; i <= N - M; i++) {
        int j=0;
        while (j<M && text[i+j]==pattern[j])
            j++;
        if (j == M){//pattern[0...M-1] = text[i, i+1, ...i+M-1]
            cout << "Pattern trovato in pos " << i << endl
            return;}
    }
    cout<< " Pattern non trovato " <<endl
    return;
}
    
```



STRATEGIA BRUTE-FORCE

i	j	i+j	0	1	2	3	4	5	6	7	8	9	10
			A	B	A	C	A	D	A	B	R	A	C
4	3	7					A	D	A	C	R		
5	0	5					A	D	A	C	R		

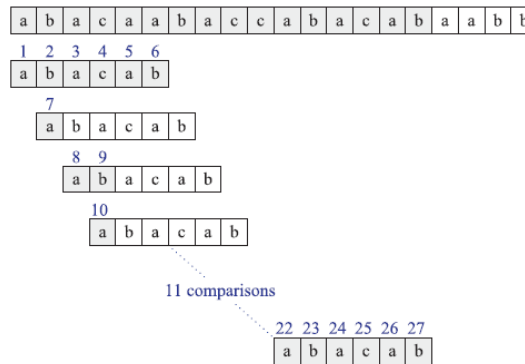
```
void pattern_matching_naive(string pattern, string text)
{
    int M = pattern.size();
    int N = text.size();
    for (int i = 0; i <= N - M; i++) {
        int j=0;
        while (j<M && text[i+j]==pattern[j])
            j++;
        if (j == M){//pattern[0...M-1] = text[i, i+1, ...i+M-1]
            cout << "Pattern trovato in pos " << i << endl
            return;}
    }
    cout<< " Pattern non trovato " <<endl
    return;
}
```

E per trovare
tutte le
occorrenze?



QUANTI CONFRONTI?

- Siano per esempio,
 $T = \text{"abacaabaccabacabaabb"}$
 $P = \text{"abacab"}$.

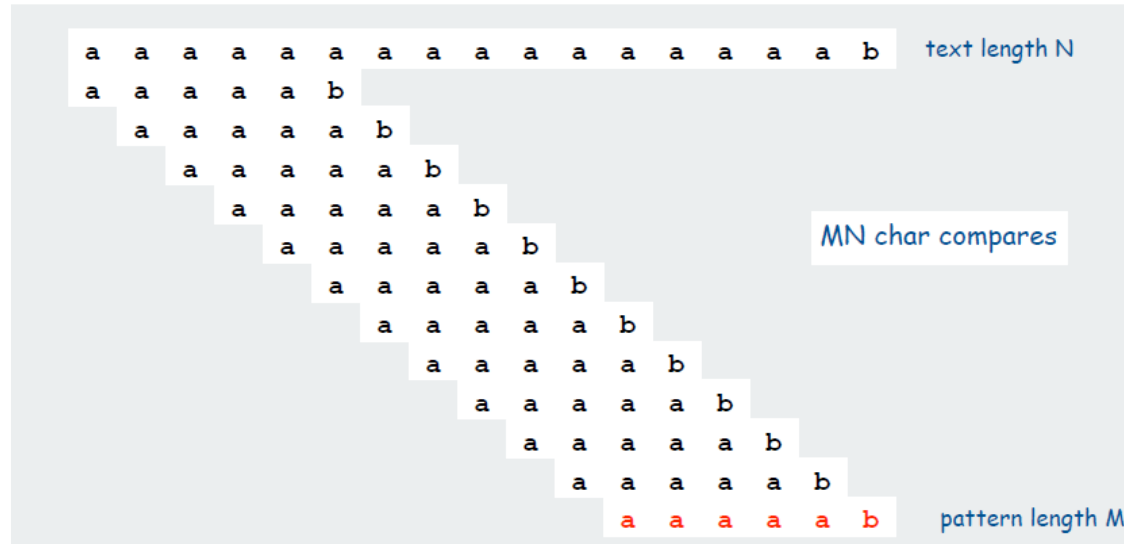


- Si effettuano 27 confronti



STRATEGIA BRUTE-FORCE: WORST CASE

- Questo approccio è lento quando il testo e il pattern sono molto ripetitivi



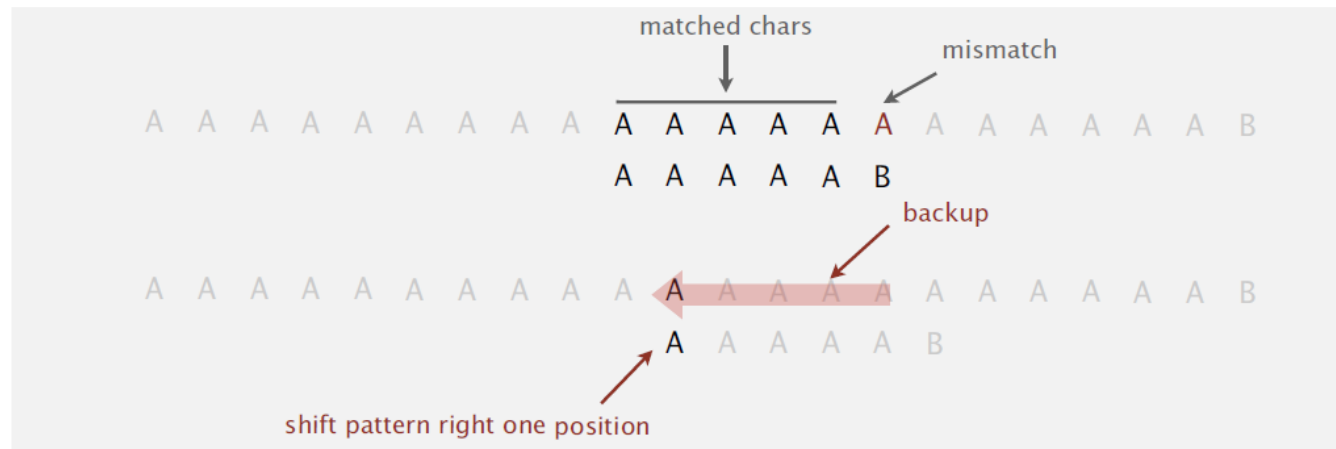
- $O(mn)$ confronti



BACKUP



- L'algoritmo brute-force necessita di effettuare numerosi backup, che oltre a essere poco convenienti spesso non si adattano al tipo di input che si ha a disposizione.



- Il metodo **find** della classe **string** usa l'approccio brute-force



OBIETTIVI

- Realizzare il pattern matching in tempo lineare
- Evitare il backup

SIAM J. COMPUT.
Vol. 6, No. 2, June 1977

FAST PATTERN MATCHING IN STRINGS*

DONALD E. KNUTH[†], JAMES H. MORRIS, JR.[‡] AND VAUGHAN R. PRATT[¶]

Abstract. An algorithm is presented which finds all occurrences of one given string within another, in running time proportional to the sum of the lengths of the strings. The constant of proportionality is low enough to make this algorithm of practical use, and the procedure can also be extended to deal with some more general pattern-matching problems. A theoretical application of the algorithm shows that the set of concatenations of even palindromes, i.e., the language $\{aa^R\}^*$, can be recognized in linear time. Other algorithms which run even faster on the average are also considered.



Don Knuth



Jim Morris

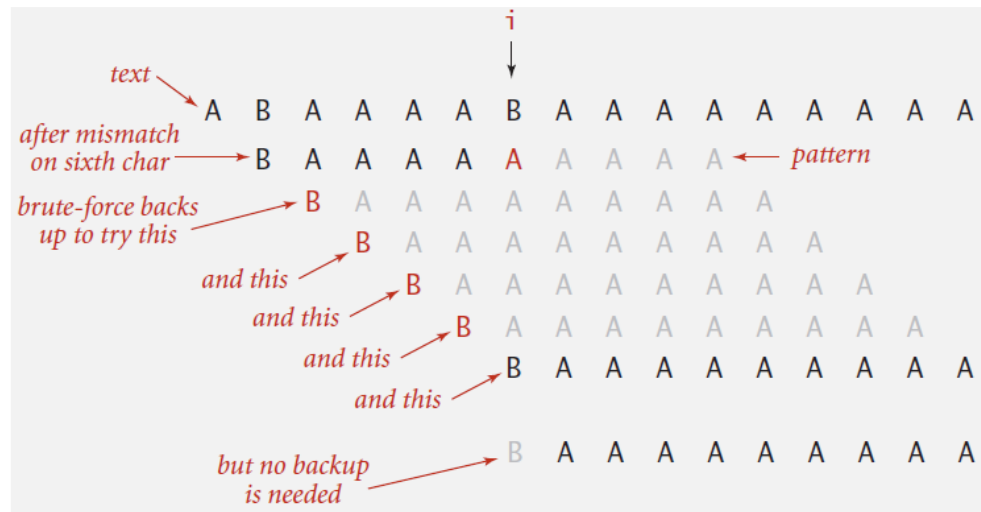


Vaughan Pratt



ALGORITMO DI KNUTH-MORRIS-PRATT (1977)

- Idea:
 - Supponiamo che stiamo cercando nel testo il pattern BAAAAAAAAA.
 - Supponiamo che abbiamo 5 match nel pattern e un mismatch in posizione 6.
 - Sappiamo che nel testo i 6 caratteri che precedono sono BAAAAB (se assumiamo che l'alfabeto è {A,B}).
 - Non serve nessun backup! Il puntatore i non deve tornare indietro! Il puntatore j del pattern può essere spostato in modo opportuno



ESEMPIO

T = BACBABAACAB ----
 ABACAB
 x
 ABACAB

P = ABACAB

Cerco il più lungo suffisso di ABAA che è anche prefisso del pattern. Qui si sposterà il pattern.



AUTOMA A STATI FINITI

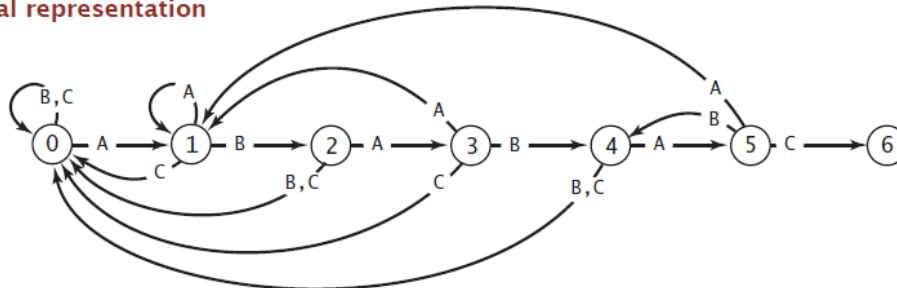
- Si usa un DFA per registrare di quanto deve tornare indietro il puntatore j quando si trova un mismatch
- Il DFA è un modello computazionale che può essere implementato usando una matrice

internal representation

j	0	1	2	3	4	5
A	1	1	3	1	5	1
B	0	2	0	4	0	4
C	0	0	0	0	0	6

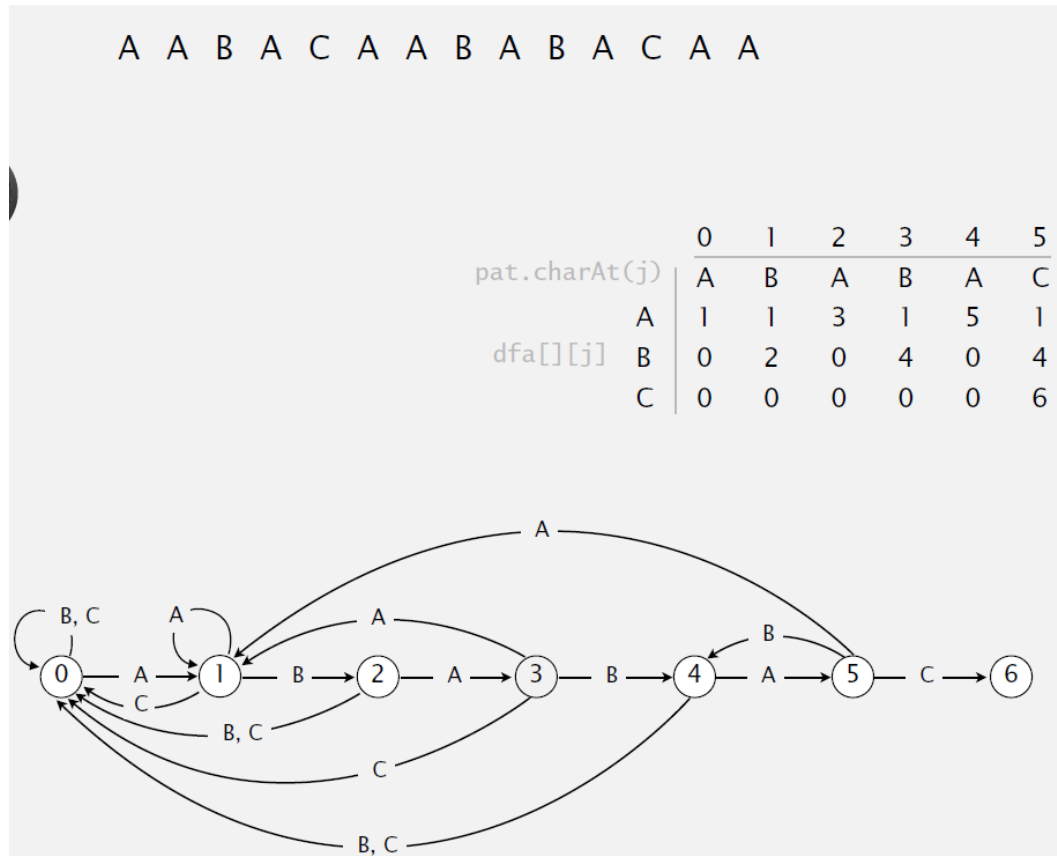
If in state j reading char c :
if j is 6 halt and accept
else move to state $\text{dfa}[c][j]$

graphical representation



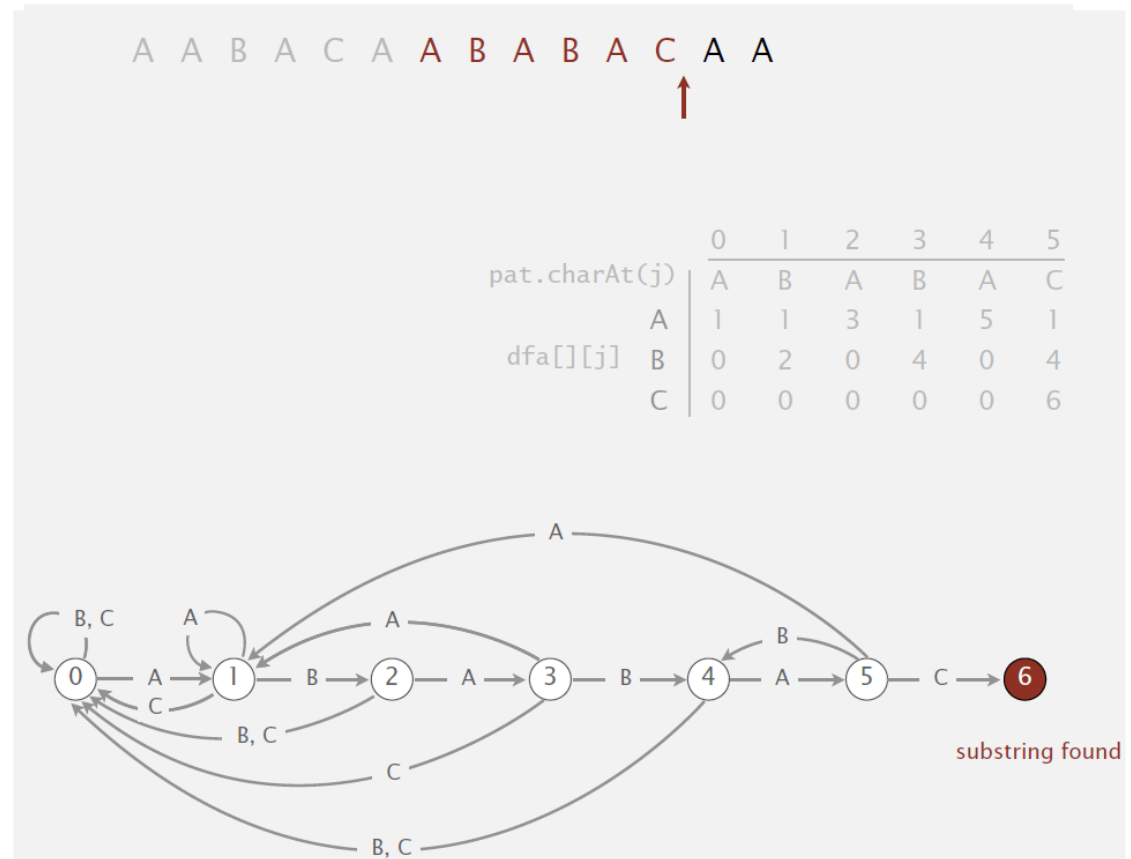
ALGORITMO KMP

- Si costruisce un opportuno DFA per il pattern P
- Si simula il DFA avendo il testo T come input



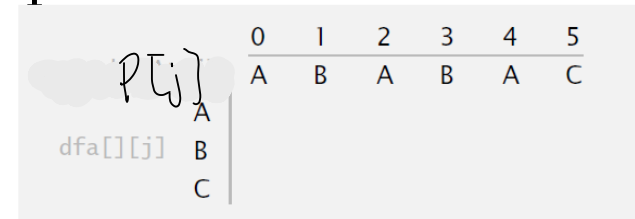
ALGORITMO KMP

- Si costruisce un opportuno DFA per il pattern P
- Si simula il DFA avendo il testo T come input



COME SI COSTRUISCE IL DFA?

- Ha uno stato per ogni prefisso P_q del pattern
- 0 è lo stato iniziale, m è lo stato finale



	0	1	2	3	4	5
A	A	B	A	B	A	C
B						
C						

- $\delta(q,a)$ è lo stato che corrisponde al più lungo prefisso di P che è suffisso di P_qa

Constructing the DFA for KMP substring search for A B A B A C



COME SI COSTRUISCE IL DFA?

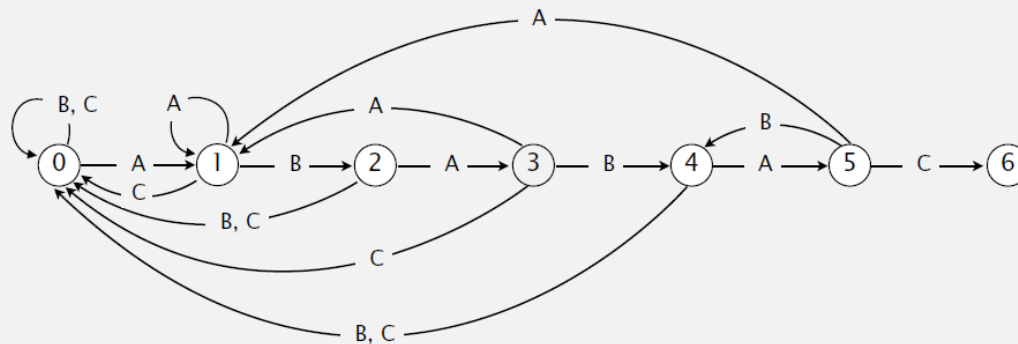
- Ha uno stato per ogni prefisso P_q del pattern

- 0 è lo stato iniziale, m è lo stato finale

$P[j]$	0	1	2	3	4	5
A	1	1	3	1	5	1
B	0	2	0	4	0	4
C	0	0	0	0	0	6

- $\delta(q,a)$ è lo stato che corrisponde al più lungo prefisso di P che è suffisso di P_qa

Constructing the DFA for KMP substring search for A B A B A C



COSTRUZIONE DELL'AUTOMA

- Per ogni stato j
 - Copia $dfa[][X]$ a $dfa[][j]$ per i casi di mismatch.
 - Assegna $dfa[pattern(j)][j]$ a $j+1$ per i casi di match.
 - Aggiorna X , che rappresenta lo stato che corrisponde al più lungo suffisso del $pattern[0,j]$ che è anche prefisso.

// build DFA from pattern

```
int m = pattern.length;
int**dfa=new int*[ALPH];
    for (int i=0;i<ALPH;i++)
        dfa[i]=new int[m];
dfa[pattern[0]][0] = 1;
for (int x = 0, j = 1; j < m; j++) {
    for (int c = 0; c < ALPH; c++)
        dfa[c][j] = dfa[c][x]; // Copy mismatch cases.
    dfa[pattern[j]][j] = j+1; // Set match case.
    x = dfa[pattern[j]][x]; // Update restart state.
}
```

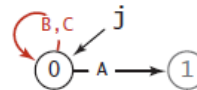
- Running Time: m accessi al pattern. Tempo e spazio proporzionale a $ALPH*m$



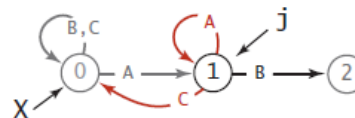
ESEMPIO DI COSTRUZIONE DEL DFA

■ P=ABABAC

	j	0
pat.charAt(j)	A	
dfa[][j]	A	1
	B	0
	C	0



	j	0	1
	X		
pat.charAt(j)	A	B	
dfa[][j]	A	1	1
	B	0	2
	C	0	0

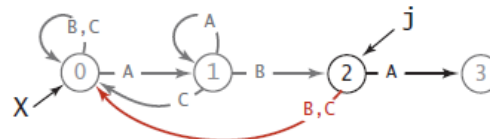


copy dfa[][X] to dfa[][j]

$dfa[pattern[j]][j] = j+1$

$X = dfa[pattern[j]][X]$

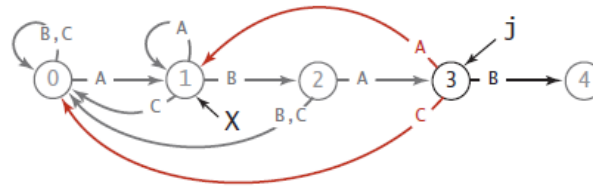
	j	0	1	2
	X			
pat.charAt(j)	A	B	A	
dfa[][j]	A	1	1	3
	B	0	2	0
	C	0	0	0



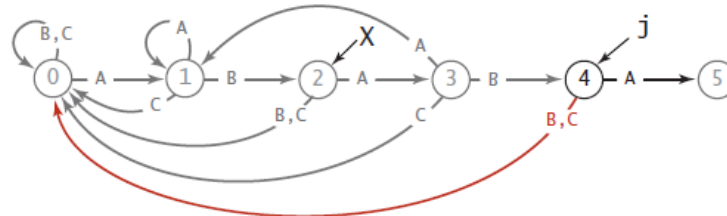
ESEMPIO DI COSTRUZIONE DEL DFA

■ P=ABABAC

j	0	1	2	3
pat.charAt(j)	A	B	A	B
dfa[][j]	A	1	1	3
	B	0	2	0
	C	0	0	0



j	0	1	2	3	4
pat.charAt(j)	A	B	A	B	A
dfa[][j]	A	1	1	3	1
	B	0	2	0	4
	C	0	0	0	0



j	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C
dfa[][j]	A	1	1	3	1	5
	B	0	2	0	4	0
	C	0	0	0	0	6

